

Technical Documentation

RAG-Based Customer Support Assistant

1. Introduction

This document explains the implementation of a RAG-based customer support assistant that answers questions from a PDF knowledge base. The system retrieves relevant support information before producing an answer, which improves grounding and reduces unsupported responses.

What is RAG?

Retrieval-Augmented Generation is an architecture where external knowledge is retrieved first and then used as context for answer generation. Instead of relying only on a model, the assistant answers using documents from the knowledge base.

Why RAG is Needed

- Customer answers must be consistent with company policies.
- Support teams need quick retrieval from documentation.
- RAG provides traceability through source chunks.
- It reduces hallucination compared with free-form generation.

Use Case Overview

The project is designed for customer support questions covering account management, billing, orders, delivery, technical issues, refunds, and returns.

2. System Architecture Explanation

The system has two major phases: ingestion and query processing.

Ingestion Phase

1. PDF is read from the data folder.
2. Text is extracted page by page.
3. Text is split into chunks.
4. Each chunk is embedded.
5. Chunks, vectors, and metadata are stored in ChromaDB.

Query Phase

1. User enters a question.
 2. Router identifies the support intent.
 3. Retriever finds relevant chunks from ChromaDB.
 4. HITL module decides if automation is safe.
 5. Answer generator returns a grounded response or escalation message.
- The User Interface (CLI) captures the user query and sends it to the query processing pipeline.

- The Router (Intent Detection Layer) analyzes the query and determines the appropriate category (e.g., billing, orders, technical support).
- The Retriever interacts with the Vector Database (ChromaDB) to fetch top-k relevant chunks using vector similarity search.
- The Embedding System ensures both documents and queries are represented in the same vector space.
- The Workflow Orchestration (LangGraph) controls execution flow between nodes.
- The HITL module evaluates confidence, query type, and safety conditions to decide whether to proceed automatically or escalate.

The Answer Generator produces a grounded response using retrieved context or returns an escalation message.

3. Design Decisions

Chunk Size Choice

A chunk size of 900 characters with 150 characters overlap is used. This keeps related Q&A pairs together while preventing chunks from becoming too large.

Embedding Strategy

The project uses deterministic local embeddings for stable execution without requiring an external API. This supports reliable evaluation in offline environments.

Retrieval Approach

The retriever performs top-k vector search in ChromaDB. It returns matching chunks and confidence information used by the HITL logic.

Prompt and Answer Design

The answer generator uses retrieved customer support text as the source of truth. For Q&A style documents, it extracts the most relevant direct answer when possible.

4. Workflow Explanation

LangGraph is used to implement workflow control. The system is modeled as a graph with defined nodes, edges, and state transitions.

Nodes:

- Input Node → receives user query
- Router Node → detects intent
- Retriever Node → fetches relevant chunks from ChromaDB
- HITL Node → evaluates whether automation is safe
- Generator Node → generates response using retrieved context
- Output Node → returns final answer

Workflow Flow:

Input → Router → Retriever → HITL → Generator → Output

State Transitions:

- Query enters system → intent is assigned
- Relevant context is retrieved → confidence is evaluated
- Decision is made → either generate response or escalate

This graph-based workflow ensures modular execution, transparency, and controlled decision-making.

5. Conditional Logic

- Order queries are routed to orders_delivery.
- Refund and payment queries are routed to billing_payments.
- Password and login queries are routed to account_management.
- App and website issues are routed to technical_support.
- Sensitive terms route to escalation.
- Vague input routes to clarification or human review.

6. HITL Implementation

The Human-in-the-Loop (HITL) module ensures that the system does not generate unreliable or unsafe responses.

When Escalation is Triggered:

- Low retrieval confidence
- No relevant chunks found
- Ambiguous or vague queries
- Sensitive queries (e.g., legal issues, complaints, fraud)

What Happens After Escalation:

- The query is flagged for human review
- The system returns a message indicating escalation
- (Optional) The query can be forwarded to a ticketing or support system

Benefits:

- Improves reliability and trustworthiness
- Prevents incorrect or harmful responses

Limitations:

- Increases response time
- Requires human intervention for completion

7. Challenges and Trade-offs

Trade-off	Explanation
Retrieval accuracy vs speed	Higher top-k can improve recall but may add irrelevant context.
Chunk size vs context quality	Large chunks preserve context but may reduce retrieval precision.
Offline reliability vs model quality	Local embeddings are reliable, while API embeddings may be semantically stronger.
Automation vs safety	HITL reduces risk but may increase response time.

8. Testing Strategy

- Run `python main.py ingest` to verify PDF extraction and ChromaDB indexing.
- Ask order, refund, password, and technical questions to verify routing.
- Ask sensitive questions to verify HITL escalation.
- Use `--debug` to inspect route, confidence, and source metadata.

Sample Queries

- How can I track my order?
- I forgot my password. What should I do?
- How do I request a refund?
- The app is not loading. What should I do?
- I want to file a legal complaint.

9. Future Enhancements

- Add web interface using Flask or FastAPI.
- Use production embedding models for stronger semantic retrieval.
- Connect HITL escalation to a ticketing system.
- Add chat memory for follow-up questions.
- Support multiple knowledge base PDFs with document filters.
- Introduce a feedback loop to continuously improve retrieval quality and response accuracy based on user interactions